

Signals in Godot 4

Entkopplung am Beispiel

1. Was sind Signals?

Ein Signal ist ein Mechanismus, mit dem ein Knoten anderen Knoten mitteilen kann, dass ein Ereignis eingetreten ist — ohne zu wissen, wer zuhört. Signals sind Godots zentrales Werkzeug zur Entkopplung von Spiellogik.

Knotentypen und ihre Signale

In Godot bestimmt der Knotentyp, welche eingebauten Signale zur Verfügung stehen. Das Ei wurde bewusst als Area2D angelegt.

Knotentyp	Relevante Signale	Verwendung
Area2D	body_entered, body_exited, area_entered	Sammelbare Objekte, Trigger-Zonen, Sensoren
StaticBody2D	input_event mouse_entered visibility_changed	Anklickbare Objekte: Türen, Truhen, NPCs, Hover-Effekte, Tooltips

Nur Area2D meldet aktiv, wenn ein anderer Körper seinen Kollisionsbereich betritt. Deshalb ist Area2D ein passender node für sammelbare Objekte.

Bevor Signale etwas ‚verdrahten‘ können, braucht es ein paar Bausteine zum Verdrahten.

Szenenaufbau / Voraussetzungen für das Beispiel

- world.tscn — die Spielwelt mit einer TileMapLayer als Terrain
- player.tscn — der Spieler als CharacterBody2D mit dem Namen „Girl“
- egg.tscn — ein Ei-Objekt (Area2D + Sprite2d, Collisionshape2D)
- hud.tscn — die Benutzeroberfläche mit einem Zähler-Label
- events.gd — ein globaler Signal-Bus als Autoload registriert (.gd Datei)

1.1 Das Problem der direkten Referenz

Ohne Signals müsste jedes Objekt eine direkte Referenz auf seine Abhängigkeiten halten. Das Ei müsste z.B. wissen, wo das HUD im Szenenbaum liegt:

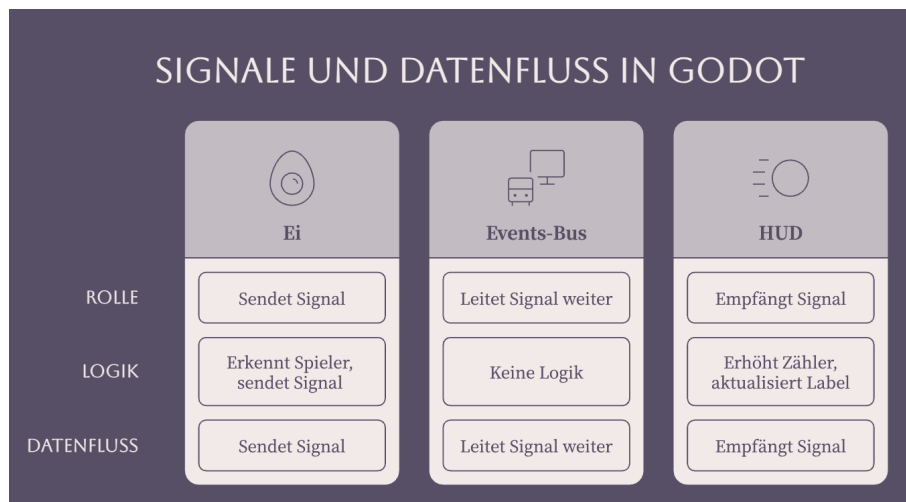
```
# Direkte Referenz — eng gekoppelt, fragil
get_node("/root/Game/HUD/CanvasLayer/EggCounter").text = "Eggs: 1"
```

Problem: Ändert sich der Szenenbaum, bricht der Pfad → unerwünschte Kopplung

2 Die Signal-Lösung

Mit einem Signal sendet das Ei nur eine Nachricht aus. Wer diese empfängt und was er damit macht, ist vollständig dem Empfänger überlassen:

- Sender (Ei): „Ich wurde eingesammelt.“
- Empfänger (HUD): „Gut, ich erhöhe den Zähler.“
- Verbindung: einmalig per `connect()` in `_ready()` hergestellt
- Signal Bus wird im Folgenden erläutert



Eingebaute vs. benutzerdefinierte Signals

- **Eingebaute Signale:** sind vordefiniert vom Knotentyp (z.B. `body_entered` bei `Area2D`)
- **Benutzerdefinierte Signale:** sind benutzerdefiniert, selbst deklariert mit dem Schlüsselwort `signal`

3. Fall A: Szeneninternes Signal

Das erste Signal bleibt vollständig innerhalb von `egg.tscn`. Das Ei reagiert selbständig auf die Spieler:innenberührung und löscht sich aus der Szene.

3.1 Das Signal `body_entered`

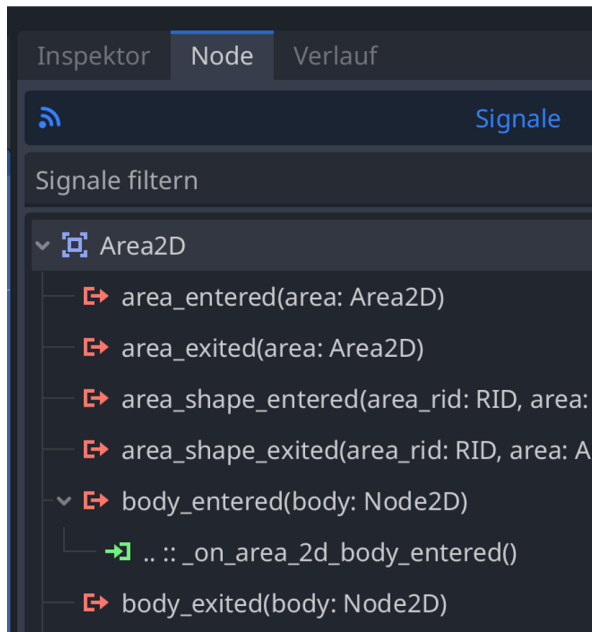
`body_entered` ist ein eingebautes Signal von `Area2D`. Es wird automatisch ausgelöst, sobald ein `PhysicsBody2D` (z.B. `CharacterBody2D`) den Kollisionsbereich des `Area2D` betritt. Das Signal übergibt den Körper, der eingetreten ist als Parameter (`body: Node2D`).

3.2 Verbindung in `_ready()`

Die Verbindung zwischen Signal und Empfängerfunktion wird in `_ready()` per Code hergestellt:

```
func _ready() -> void:
    body_entered.connect(_on_body_entered)
```

Alternativ kann diese Verbindung im Godot-Editor über den Node-Reiter (Signals) per Mausklick hergestellt werden (siehe screenshot). Godot schreibt dann eine [connection]-Zeile in die .tscn-Datei.



3.3 Die Empfängerfunktion

```
func _on_body_entered(body: Node2D) -> void:
    if body.name == "Girl":
        Events.egg_collected.emit()
        queue_free()
```

Erklärung Zeile für Zeile:

- Name der Funktion wurde in `_ready()` definiert
- `body.name == "Girl"`: Sicherheitsprüfung, nur die Spieler:in darf das Ei einsammeln
- `Events.egg_collected.emit()`: Das Ei sendet das Signal über den globalen Bus (dazu mehr in Abschnitt 4)
- `queue_free()`: Das Ei löscht sich am Ende des aktuellen Frames sicher aus dem Szenenbaum. Sofortiges `free()` wäre unsicher, da intern der Node noch referenziert wird, d.h. ‚schwebender Zeiger‘ kann undefiniertes Verhalten erzeugen.

4. Fall B: Szenenübergreifendes Signal

Das Ei und das HUD befinden sich in verschiedenen Ästen des Szenenbaums. Eine direkte Verbindung per `connect()` ist möglich, aber aufwändig. Die elegantere Lösung ist ein globaler Signal-Bus.

4.1 Das Problem der Szenengrenzen

egg.tscn wird als Instanz in world.tscn eingebunden. hud.tscn ist in game.tscn eingebunden. Beide kennen sich nicht. Eine direkte Verbindung müsste im übergeordneten Knoten eingerichtet werden.

4.2 Der Signal-Bus: events.gd als Autoload

events.gd ist als Autoload („Globals“) in den Projekteinstellungen registriert, d.h. der Knoten Events existiert global, bevor die erste Szene geladen wird, und ist von jedem Skript im Projekt aus erreichbar.

In events.gd werden alle projektübergreifenden Signals deklariert:

```
# scripts/events.gd
extends Node

signal egg_collected
```

Der Bus* selbst hat **keine Logik** — er ist nur ein zentraler Verteiler für Nachrichten. ‚Busbegriff‘ kommt aus der Elektrotechnik (no point-to-point connections)

4.3 Emit: Das Ei sendet

Wenn der Spieler ein Ei berührt, sendet das Ei das Signal über den Bus:

```
Events.egg_collected.emit()
```

Das Ei kennt das HUD nicht. Es weiß nur: „Ich wurde eingesammelt, und ich melde das dem Bus.“

4.4 Connect: Das HUD hört zu

Das HUD verbindet sich in `_ready()` mit dem Signal auf dem Bus:

```
# scenes/hud.gd
extends Control

var eggs_collected: int = 0

func _ready() -> void:
    Events.egg_collected.connect(_on_egg_collected)
    $CanvasLayer/EggCounter.text = "Eggs: 0"

func _on_egg_collected() -> void:
    eggs_collected += 1
    $CanvasLayer/EggCounter.text = "Eggs: %d" % eggs_collected
```

Erklärung der Formatierung: „Eggs: %d“ ist ein Formatstring. %d steht als Platzhalter für eine ganze Zahl (integer). Der %-Operator ersetzt den Platzhalter mit dem Wert von `eggs_collected`.

5. Zusammenfassung

Die folgende Tabelle zeigt den vollständigen Datenfluss vom Moment der Kollision bis zur Anzeige im HUD:

Der Datenfluss verläuft in drei Stationen:

- **1. Ei:** Das Ei (Area2D) erkennt die Spieler:in über `body_entered`, sendet `Events.egg_collected.emit()` und löscht das ‚Ei‘ anschließend.
- **2. Events-Bus:** Der Events-Bus leitet das Signal weiter, ohne selbst Logik auszuführen.
- **3. HUD:** Das HUD empfängt das Signal, erhöht den Zähler und aktualisiert das Label.